

1ª Entrega

Academico

Chatbot RAG para o curso de Ciencia de Dados e Machine Learning - CEUB

Autor	Matheus Alves
Curso	Ciencia de Dados e Machine Learning (CDML)
Instituicao	CEUB - Centro Universitario de Brasilia
Semestre	1o semestre de 2025
Data	14 de maio de 2026
Tipo	Material de apoio para apresentacao

Este documento descreve a arquitetura, capacidades e limitacoes do sistema **Academico** - um assistente conversacional construido com tecnicas de Retrieval-Augmented Generation (RAG) e consulta a banco relacional para apoiar estudantes ao longo da jornada academica.

1. Visao geral

Academico e um chatbot que ajuda o aluno do curso de Ciencia de Dados e Machine Learning do CEUB a tirar duvidas sobre o conteudo das disciplinas e a acompanhar sua vida academica - proximas provas, prazos de entrega, notas e frequencia. O sistema combina duas tecnicas: **Retrieval-Augmented Generation** (RAG) sobre o material textual das materias e **consulta direta a um banco relacional** com os dados estruturados do aluno. Um modelo de linguagem (LLM) recebe esses dois contextos e gera respostas em linguagem natural via streaming.

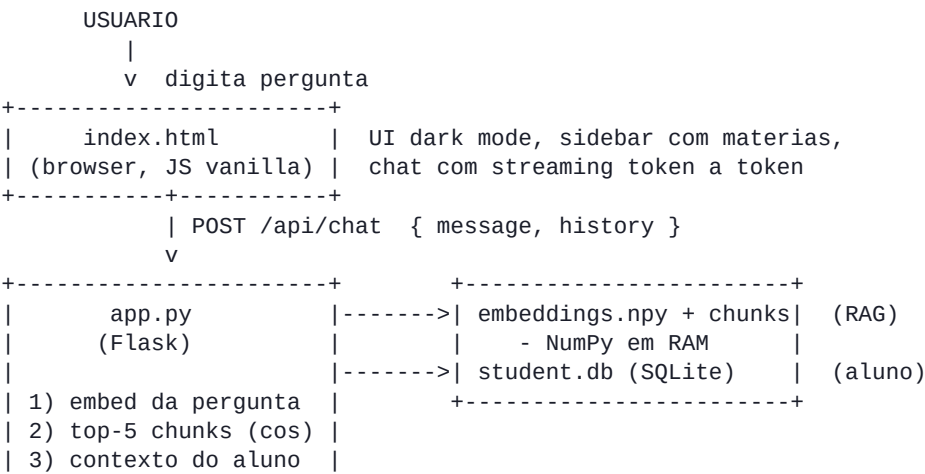
O objetivo desta primeira entrega e demonstrar a arquitetura completa funcionando ponta a ponta: ingestao da base de conhecimento, persistencia dos dados do aluno, busca semantica, integracao com LLM e interface web utilizavel.

2. Stack tecnologica

Camada	Tecnologia	Funcao
Frontend	HTML + CSS + JS vanilla + marked.js	Interface dark mode com chat em streaming
Backend	Python 3.10+ com Flask 3	API REST (/, /api/student, /api/chat)
Embeddings	sentence-transformers (MiniLM 384d)	Vetorizacao local da base e das queries
Vetor store	NumPy em memoria	Busca por similaridade do cosseno
Dados estruturados	SQLite	Aluno, disciplinas, matriculas, atividades, notas, frequencia
LLM (chat)	OpenRouter (gpt-4o-mini padrao)	Geracao das respostas em streaming SSE
Configuracao	python-dotenv (.env)	Chaves de API e parametros do sistema

3. Arquitetura

O sistema e construido sobre tres pilares: uma camada de recuperacao semantica (RAG) que entende o significado das perguntas, uma camada de dados estruturados que conhece o estado real do aluno, e um LLM que sintetiza ambos em uma resposta natural. O diagrama abaixo mostra o fluxo:



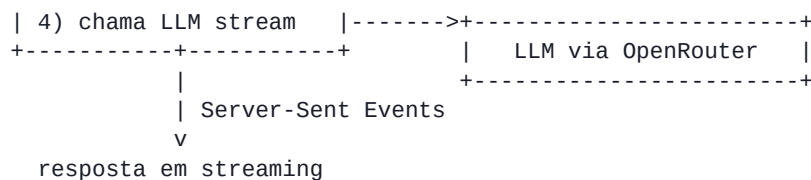


Figura 1. Fluxo arquitetural do sistema.

Fluxo detalhado de uma mensagem

1. O frontend envia a pergunta do usuario e o historico recente para **/api/chat**.
2. O backend gera o **embedding** da pergunta usando o modelo local de sentence-transformers.
3. Compara o vetor da pergunta com os 62 vetores da base via produto interno em vetores normalizados (equivalente matematico a similaridade do cosseno).
4. Seleciona os **5 trechos mais similares** com score acima de 0,20.
5. Consulta o SQLite para montar um bloco de contexto com matriculas, proximas atividades, notas e frequencia do aluno.
6. Monta o prompt final: *system prompt + dados do aluno + chunks recuperados + historico + pergunta*.
7. Chama o LLM com **stream=True** e repassa os tokens ao frontend via Server-Sent Events.
8. Ao final, envia tambem as **fontes consultadas** (arquivo, secao, score), exibidas no UI.

4. Base de conhecimento

A base e composta por **10 arquivos Markdown** organizados em data/knowledge/. A divisao em chunks e feita por secao do Markdown (separador ##), com fallback para janela deslizando quando uma secao excede 2.200 caracteres.

Arquivo	Conteudo	Chunks
institucional.md	CEUB, curso, regras gerais, frequencia, calendario	9
grade-curricular.md	Visao geral dos 7 semestres, pre-requisitos, trilhas	10
processos-academicos.md	Aprovacao, trancamento, aproveitamento, TCC	10
disciplinas-1-semester.md	ALGA, BDI, BOOTI, ES, ICD, LP (detalhado)	6
disciplinas-2-semester.md	BDII, BII, CC, DCDI, FSC	5
disciplinas-3-semester.md	BDN, BOOTII, BIII, DCDII, ECD, PE	6
disciplinas-4-semester.md	AMML, FBD, OACD, RI	4
disciplinas-5-semester.md	APDL, BD NoSQL, BI, PI I	4
disciplinas-6-semester.md	APS, DS IA, PI II, VD	4
disciplinas-7-semester.md	ED, PLN, PI III, SD	4
TOTAL		62

Cada chunk tem em media **445 caracteres** (minimo 32, maximo 977). Os 62 vetores formam uma matriz 62 x 384 que cabe em memoria sem problema - busca em milissegundos.

5. Dados estruturados do aluno

O banco SQLite (data/student.db) tem 6 tabelas: **alunos** (perfil), **disciplinas** (catalogo das 31 disciplinas), **matriculas** (vinculo aluno-disciplina por semestre), **atividades** (provas, trabalhos, listas, projetos com data, peso e status), **notas** (notas das atividades corrigidas) e **frequencia** (aulas dadas e faltas).

Snapshot atual (aluno: Matheus Alves)

Semestre: 1o **Matriculado em:** ALGA, BDI, BOOTI, ES, ICD, LP

Proximas atividades pendentes:

Data	Tipo	Disciplina	Peso	Descricao
10/05	LISTA	LP	10%	Estruturas de repeticao (while/for)
13/05	PROVA	ALGA	40%	P1 - Vetores, matrizes, determinantes
15/05	TRABALHO	ICD	30%	EDA com dataset Titanic em Jupyter
17/05	PROVA	BDI	40%	P1 - Modelagem ER e SQL basico

Data	Tipo	Disciplina	Peso	Descricao
20/05	TRABALHO	ES	20%	Diagrama de casos de uso
23/05	PROVA	LP	40%	P1 - Logica e estruturas basicas
28/05	PROJETO	BOOTI	25%	Entrega 1 - Problema + dataset

Frequencia: alerta em **Engenharia de Software (ES)** com 16,7% de faltas - proximo do limite de 25% (que reprova por falta). Demais disciplinas dentro da margem segura.

6. O que o chat responde

O sistema responde tres tipos de pergunta, classificados pela natureza do dado consultado. Cada categoria mostra o real valor de combinar RAG com dados estruturados.

6.1. Perguntas de conteudo (puro RAG)

Duvidas sobre o material das disciplinas. O bot busca trechos relevantes na base de conhecimento e responde com base neles. *Cobre todas as 31 disciplinas do curso.*

Pergunta do aluno	O que o bot consulta
Explica produto escalar com exemplo	Trechos de ALGA
Diferenca entre 1FN, 2FN e 3FN	Trechos de BDI (normalizacao)
Como funciona o CRISP-DM?	Trechos de ICD
O que e o metodo agil Scrum?	Trechos de ES
Quando vou usar calculo no curso?	Trechos de CC, AMML, OACD
Como funciona PCA?	Trechos de AMML / ALGA
O que e RAG?	Trechos de RI / PLN (meta!)

6.2. Perguntas de calendario/situacao (puro SQL)

Duvidas sobre o estado do aluno: provas marcadas, notas, frequencia. O bot consulta o SQLite diretamente.

Pergunta do aluno	Resposta esperada
Quais provas eu tenho essa semana?	Lista P1 de ALGA (13/05) e P1 de BDI (17/05)
Como ta minha frequencia em ES?	4 faltas em 24 aulas (16,7%) - atencao ao limite
Quais minhas notas ate agora?	Lista as 6 listas corrigidas com valores
Em quais disciplinas estou matriculado?	Lista as 6 do 1o semestre
Qual o peso da proxima entrega?	Lista 3 de LP, dia 10/05, peso 10%

6.3. Perguntas hibridas (RAG + SQL)

Aqui o sistema mostra seu valor real. O bot combina dados estruturados com conteudo da base para responder de forma personalizada - algo que nem RAG puro nem SQL puro conseguem.

Pergunta do aluno	O que o bot faz
Tenho prova de ALGA daqui 5 dias, me ajuda a revisar	Consulta data via SQL; busca topicos em ALGA via RAG; monta plano de revisao
Qual materia to pior e o que estudar?	Calcula menor media via SQL; busca conteudo via RAG; sugere estudo focado
Minha proxima prova de BDI cobre o que?	SQL retorna P1 de BDI; RAG traz explicacao de ER e SQL basico
Me da um plano de estudos pra essa semana	SQL lista 7 atividades pendentes; RAG traz conteudo de cada; bot organiza cronograma

Pergunta do aluno	O que o bot faz
Posso faltar amanhã em ES?	SQL mostra 16,7%; RAG traz regra de 25%; bot calcula e alerta

7. Comportamento do bot (system prompt)

O comportamento do bot é controlado por um **system prompt** explícito - um conjunto de regras inseridas em toda chamada ao LLM antes da pergunta do usuário. Isso reduz alucinação e mantém o tom consistente.

1. **Anti-alucinação:** baseia-se EXCLUSIVAMENTE no contexto fornecido. Se não houver informação suficiente, diz 'não tenho essa informação'.
2. **Datas e notas:** NUNCA inventa datas, notas, prazos ou nomes de disciplinas. Sempre cita o que está nos dados estruturados.
3. **Tom:** português brasileiro, acolhedor e direto - como veterano ajudando calouro.
4. **Formato:** markdown (negrito, listas, blocos de código) para clareza.
5. **Especificidade:** sempre cita a disciplina ou data ao responder.
6. **Pro-atividade:** alerta sobre prazos curtos, frequência crítica, notas baixas.
7. **Fallback:** em dúvida, indica procurar coordenação ou professor.

8. Funcionalidades implementadas

- Chat web responsivo com **dark mode**
- Sidebar dinâmica com perfil do aluno, disciplinas matriculadas e próximas atividades coloridas por urgência (verde / amarelo / vermelho)
- **Streaming token a token** via Server-Sent Events - resposta aparece em tempo real
- Renderização Markdown nas respostas (listas, negrito, blocos de código)
- Indicação visual das **fontes consultadas** (chunks de origem + score de similaridade) abaixo de cada resposta
- Histórico de conversa mantido no frontend e enviado ao backend nas últimas 6 mensagens
- **Prompts sugeridos** clicáveis na tela inicial
- **Atalhos:** clicar em uma disciplina ou atividade gera prompt pre-preenchido
- Busca vetorial em memória com filtro de score mínimo (0,20)
- Backend configurável: troca de provedor LLM e modelo via `.env` sem mexer no código
- Suporte a **dois backends de embedding**: local (sentence-transformers) e OpenAI (text-embedding-3-small)

9. Limitações conhecidas

São limitações reconhecidas que **não comprometem** a 1ª entrega mas serão abordadas em iterações seguintes:

- **Single-user:** todo o sistema assume um único aluno (id=1). Multiusuário exige autenticação e isolamento de dados.
- **Histórico não persistido:** as conversas só vivem na sessão do navegador. Recarregar a página perde o contexto.

- **Sem re-ranking:** a busca usa top-5 cru, sem segunda passada por LLM para reordenar. Funciona bem na base atual mas pode degradar com bases maiores.
- **Sem avaliacao automatica:** ainda nao ha metricas medindo qualidade do retrieval (precision@k, MRR). Material para a proxima entrega.
- **Datas codificadas no seed:** as atividades sao geradas com data_entrega relativa a 'hoje'. Em producao essas datas viriam do sistema academico da instituicao.

10. Proximos passos (entregas seguintes)

1. **Persistir conversas** em SQLite para manter contexto entre sessoes.
2. Implementar **re-ranking** com o proprio LLM para melhorar precisao da busca.
3. Adicionar conjunto de teste anotado e calcular **metricas de retrieval** (precision@k, recall@k, MRR).
4. Comparar pelo menos 3 modelos de embedding (MiniLM, multilingual-e5-large, BGE-M3) com metricas concretas - material direto para Probabilidade e Estatistica e Aprendizagem de Maquina.
5. Estudo de impacto da estrategia de **chunking** (por secao vs janela deslizante vs por unidade) na qualidade do retrieval.
6. Roteamento explicito de intencao (conteudo / calendario / hibrido) com classificador.
7. Deploy em producao (Vercel ou Render) - README ja traz as instrucoes.

11. Estrutura de arquivos

```
.
|-- app.py                # backend Flask (rotas /, /api/student, /api/chat)
|-- ingest.py             # gera embeddings da base de conhecimento
|-- embeddings.py         # camada de embedding (local ou OpenAI)
|-- seed_db.py            # cria e popula o SQLite do aluno
|-- requirements.txt      # dependencias Python
|-- .env                  # configuracao local (nao vai pro repositorio)
|-- .env.example          # template documentado das variaveis
|-- .gitignore
|-- README.md             # instrucoes de setup e arquitetura
|-- ESPECIFICACOES.md    # este documento em Markdown
|-- data/
|   |-- knowledge/        # 10 arquivos .md com conteudo das disciplinas
|   |-- chunks.json       # 62 chunks com texto e metadados
|   |-- embeddings.npy    # matriz NumPy 62x384 normalizada
|   |-- student.db        # banco do aluno (6 tabelas)
'-- templates/
    |-- index.html        # interface completa (CSS + JS embutidos)
```

12. Como executar

Cinco comandos sequenciais. Apenas o passo 4 precisa de internet (primeiro download do modelo de embedding, 120MB).

```
python -m venv .venv
.venv\Scripts\activate          # Windows
pip install -r requirements.txt  # ~5-10 min na primeira vez
python seed_db.py               # popula o banco do aluno
python ingest.py                # baixa modelo e gera embeddings
python app.py                   # sobe em http://localhost:5000
```

O .env ja esta configurado com a chave OpenRouter e o modelo openai/gpt-4o-mini. Para trocar de modelo (Llama 3.3 70B gratuito, Gemini Flash, Claude Haiku), basta editar a variavel CHAT_MODEL.

Obrigado.

Material de apoio para a apresentacao da 1a entrega.

Matheus Alves · CDML · CEUB · 14/05/2026